A Tour of Deep Learning

Christian Rauch

(changed: 2026-04-24)

Scope

Topics covered:

- Neural networks and their components
- Loss functions and likelihood
- Optimisation and training
- Generalisation, bias–variance trade-off, model selection
- Regularisation and data augmentation

Coming soon:

- Architectures: CNNs, RNNs, ResNets, Transformers, GNNs
- Paradigms: unsupervised learning, reinforcement learning
- Generative models: VAEs, normalising flows, diffusion models, GANs

Intended as a self-contained starting point; pointers to further reading are given in the references.

Basics

Machine Learning Paradigms

Supervised Learning

Learn a mapping from inputs to outputs using labelled data.

- **Regression** – predict continuous values
- **Classification** – predict discrete categories

Unsupervised Learning

Discover structure in unlabelled data.

- **Generative models** – learn the data distribution $p(\mathbf{x})$
- **Latent variables** – infer hidden representations $\mathbf{z}$ from observations $\mathbf{x}$

Reinforcement Learning

An agent learns to act in an environment by maximising cumulative reward through trial and error.

Neural Networks

A **neural network** is a parametric function built by stacking simple computational units called *neurons*.

For a **single neuron**, the pre-activation z, bias b, and output h are scalars. The neuron computes a weighted sum of its inputs, adds a bias, and applies a non-linear activation:

$$z = \mathbf{w}^\top \mathbf{x} + b, \qquad h = \sigma(z)$$

A network arranges many such units into layers:

- **Input layer**: receives the feature vector $\mathbf{x}$
- **Hidden layers**: transform the representation step by step
- **Output layer**: produces the final prediction $\hat{\mathbf{y}}$

By composing linear transformations with non-linear activations, neural networks can represent highly complex functions and learn useful features directly from data.

Multilayer Perceptron (MLP)

An **MLP** is a specific type of **feed-forward neural network** with fully connected layers.

It is a composition of L layers, each applying an affine transformation followed by a non-linear activation function σ:

$$\begin{aligned} \mathbf{h}_0 &= \mathbf{x} \\ \mathbf{h}_l &= \sigma\big(\mathbf{W}_l\,\mathbf{h}_{l-1} + \mathbf{b}_l\big), \quad l = 1, \ldots, L-1 \\ \hat{\mathbf{y}} &= \mathbf{W}_L\,\mathbf{h}_{L-1} + \mathbf{b}_L \end{aligned}$$

- $\mathbf{h}_l \in \mathbb{R}^{D_l}$: hidden representation at layer l ("features")
- $\mathbf{W}_l \in \mathbb{R}^{D_l \times D_{l-1}}$, $\mathbf{b}_l \in \mathbb{R}^{D_l}$: learnable weights and biases
- σ: activation function (e.g. ReLU), applied element-wise
- The last layer is typically linear (no activation), producing raw outputs

All parameters $\phi = \{\mathbf{W}_1, \mathbf{b}_1, \ldots, \mathbf{W}_L, \mathbf{b}_L\}$ are learned jointly by minimising $\mathcal{L}(\phi)$.

Weights and Biases

In a layer

$$\mathbf{h} = \sigma\big(\mathbf{W}\mathbf{x} + \mathbf{b}\big),$$

the **weights** and **biases** play different roles.

- **Weights W** determine how strongly each input component influences each neuron
- **Biases b** shift the pre-activations independently of the input
- Together, they define the affine transformation before the nonlinearity

Intuitively, weights control the **direction and strength** of a response, while biases control its **offset or threshold**.

Activation Functions

An **activation function** σ is applied element-wise after each linear transformation in a neural network, introducing non-linearity:

$$\mathbf{h} = \sigma\big(\mathbf{W}\mathbf{x} + \mathbf{b}\big)$$

Without activation functions, stacking linear layers collapses to a single linear map.

Common choices:

- **ReLU**: $\sigma(z) = \max(0, z)$ – piecewise linear, most widely used
- **Sigmoid**: $\sigma(z) = \frac{1}{1+e^{-z}}$ – outputs in $(0, 1)$, used for probabilities
- **Tanh**: $\sigma(z) = \tanh(z)$ – outputs in $(-1, 1)$, zero-centred
- **Leaky ReLU**: $\sigma(z) = \max(\alpha z, z)$, small $\alpha > 0$ – avoids "dead" neurons

Networks with **ReLU** activations naturally produce *piecewise linear* mappings. Adding more hidden units increases the number of linear regions, improving expressiveness.

Training a Model

A model f_ϕ with parameters ϕ maps inputs $\mathbf{x}$ to predictions $\hat{\mathbf{y}}$:

$$\hat{\mathbf{y}} = f_\phi(\mathbf{x})$$

A **loss function** $\mathcal{L}$ measures the discrepancy between predictions and targets:

$$\mathcal{L}(\phi) = \frac{1}{N} \sum_{i=1}^{N} \ell\big(f_\phi(\mathbf{x}_i),\ \mathbf{y}_i\big)$$

Training (learning, model fitting) means finding parameters ϕ^* that minimise the loss:

$$\phi^* = \arg\min_{\phi}\ \mathcal{L}(\phi)$$

This is typically solved iteratively via **gradient descent**:

$$\phi \leftarrow \phi - \alpha\, \nabla_\phi\, \mathcal{L}(\phi)$$

where α is the learning rate.

Hyperparameters

Hyperparameters are configuration choices set *before* training; they are **not learned** from data. In contrast, the model **parameters** (such as weights and biases) are learned during training by minimising the loss.

Typical examples include:

- **Learning rate** α: step size in gradient descent
- **Network size**: number of layers and hidden units
- **Batch size**: number of samples used per parameter update
- **Number of epochs**: how many passes are made over the training set

Hyperparameters strongly affect optimisation, generalisation, and runtime, so they are usually selected using validation data.

Piecewise Linear Functions (One-Dimensional)

By the **universal approximation theorem**, any continuous function $g\colon \mathbb{R}^D \to \mathbb{R}^K$ can be approximated arbitrarily well by a **piecewise linear** (PWL) function.

A PWL function partitions the input space into regions, each with its own affine map:

$$f(\mathbf{x}) = \mathbf{A}_r\mathbf{x} + \mathbf{b}_r \quad \text{for } \mathbf{x} \in \mathcal{R}_r$$

In 1-D ($D = K = 1$): the function is a chain of line segments joined at *breakpoints*:

- Each segment r has its own slope a_r and intercept b_r: $f(x) = a_r x + b_r$
- Increasing the number of segments yields a closer fit to any continuous curve
- Even a smooth function like $\sin(x)$ can be matched to arbitrary precision by choosing enough breakpoints

Piecewise Linear Functions (Multi-Dimensional)

In D dimensions: the input space $\mathbb{R}^D$ is partitioned into *convex polytopes* $\mathcal{R}_1, \ldots, \mathcal{R}_M$ (higher-dimensional analogues of intervals).

Inside each polytope, the mapping is a separate affine transformation:

$$f(\mathbf{x}) = \mathbf{A}_r \mathbf{x} + \mathbf{b}_r, \quad \mathbf{A}_r \in \mathbb{R}^{K \times D}, \ \mathbf{b}_r \in \mathbb{R}^K$$

- Each polytope is a convex region bounded by hyperplanes (edges in 2-D, faces in 3-D, ...)
- Different regions can have entirely different linear behaviours, enabling the overall function to bend and turn
- More polytopes $\Rightarrow$ finer approximation of any smooth target $g \colon \mathbb{R}^D \to \mathbb{R}^K$

Counting Linear Regions

Each ReLU unit $\max(0, z)$ introduces one hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ that splits the input space.

Single hidden layer with n units, input dimension D:

$$N_{\max} = \sum_{j=0}^{D} \binom{n}{j} = \mathcal{O}(n^D)$$

Deep network with L layers of width $n \geq D$:

$$N_{\max} \geq \left(\left\lfloor \frac{n}{D} \right\rfloor + 1 \right)^{(L-1)D} \cdot \sum_{j=0}^{D} \binom{n}{j}$$

Each layer can fold and subdivide regions created by previous layers, so depth yields *exponential* growth in expressiveness.

$\Rightarrow$ **Deeper networks** are exponentially more expressive per parameter than wider shallow ones.

Intuition: Why Depth Helps

A deep network repeatedly alternates between:

- an **affine map** $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$ that rotates, stretches, and shifts the representation
- a **nonlinearity** such as ReLU that cuts space along hyperplanes and keeps only part of each response

Geometrically, each layer can **fold** the input space so that points that were far apart become close in the hidden representation.

After several such folds, the network can:

- create many simple linear pieces
- reuse patterns detected in earlier layers
- make a complicated decision boundary look simple in the final feature space

So deep networks work well because they build complex functions by **composing many simple transformations**.

Losses and Likelihood

Constructing a Loss Function

A loss function compares the model prediction with the target and assigns a **small value** to good predictions and a **large value** to bad ones.

A common construction has three parts:

1. Make a prediction: $\hat{\mathbf{y}}_i = f_\phi(\mathbf{x}_i)$
2. Define a per-sample penalty: $\ell(\hat{\mathbf{y}}_i, \mathbf{y}_i)$
3. Aggregate over the dataset:

$$\mathcal{L}(\phi) = \frac{1}{N} \sum_{i=1}^{N} \ell\big(f_\phi(\mathbf{x}_i), \mathbf{y}_i\big)$$

Optionally, one adds a **regularisation** term to encourage simpler parameter values:

$$\mathcal{L}_{\text{total}}(\phi) = \mathcal{L}_{\text{data}}(\phi) + \lambda\, \Omega(\phi)$$

Maximum Likelihood

In supervised learning, we often model the **conditional probability**

$$p(\mathbf{y} \mid \mathbf{x}; \phi)$$

of the target $\mathbf{y}$ given the input $\mathbf{x}$.

Maximum likelihood estimation chooses parameters that make the observed training data most probable:

$$L(\phi) = \prod_{i=1}^{N} p(\mathbf{y}_i \mid \mathbf{x}_i; \phi)$$

The optimal parameters are those that maximise this likelihood:

$$\phi^* = \arg\max_{\phi} L(\phi) = \arg\max_{\phi} \prod_{i=1}^{N} p(\mathbf{y}_i \mid \mathbf{x}_i; \phi)$$

Log-Likelihood

The likelihood is a product of many probabilities in $[0, 1]$, so it can become **extremely small**.
We therefore use the **log-likelihood**, which turns products into sums:

$$\log L(\phi) = \sum_{i=1}^{N} \log p(\mathbf{y}_i \mid \mathbf{x}_i; \phi)$$

Because log is monotonic (larger L means larger $\log L$):

$$\arg\max_{\phi} L(\phi) = \arg\max_{\phi} \log L(\phi)$$

In optimisation, we usually minimise the **negative log-likelihood**:

$$\mathcal{L}_{\mathrm{NLL}}(\phi) = -\sum_{i=1}^{N} \log p(\mathbf{y}_i \mid \mathbf{x}_i; \phi)$$

$$\phi^* = \arg\min_{\phi} \mathcal{L}_{\mathrm{NLL}}(\phi)$$

Cross-entropy for classification is the negative log-likelihood for a categorical output distribution.

Non-Probabilistic View of Losses

A loss function does **not** have to come from an explicit probabilistic model.

We can also choose a loss directly as a **penalty** that encourages the behaviour we want, e.g.:

- small error for regression
- correct classification with a large margin
- robustness to outliers

Example: **support vector machines (SVMs)** use a **hinge loss**, which does not come from a probabilistic model but directly encourages a separating margin.

In this view, the goal is simply:

$$\phi^* = \arg\min_{\phi} \mathcal{L}(\phi)$$

Prediction Task Types

The choice of loss depends on the type of target variable y.

Common supervised prediction tasks include:

- **Regression**: predict a continuous value
- **Classification**: predict a discrete label

There are also more specialised cases, for example:

- **Count prediction**: non-negative integers
- **Ordinal prediction**: ordered categories
- **Structured prediction**: sequences, trees, segmentation masks

Regression

In **regression**, the target is a continuous value rather than a discrete class label.

The model outputs a real-valued prediction

$$\hat{y} = f_{\phi}(\mathbf{x}) \in \mathbb{R}$$

We then need a loss that measures how far the prediction is from the true target value y. Common choices penalise the prediction error $\hat{y} - y$ directly.

Mean Squared Error Loss

For regression tasks, a common choice for the per-sample loss is the **squared error**:

$$\ell(\hat{y},\ y) = (\hat{y} - y)^2$$

The **Mean Squared Error** averages over all N training samples:

$$\mathcal{L}_{\text{MSE}}(\phi) = \frac{1}{N} \sum_{i=1}^{N} \left(f_\phi(\mathbf{x}_i) - y_i\right)^2$$

Properties:

- Always ≥ 0; equals 0 only for perfect predictions
- Differentiable everywhere – convenient for gradient descent
- Penalises large errors more heavily (quadratic growth)

MSE and Gaussian Likelihood

In regression, MSE arises naturally if we assume the target is generated from a **Gaussian distribution** around the prediction:

$$y_i \mid \mathbf{x}_i \sim \mathcal{N}\big(f_\phi(\mathbf{x}_i),\ \sigma^2\big)$$

Then the conditional density is

$$p(y_i \mid \mathbf{x}_i; \phi) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - f_\phi(\mathbf{x}_i))^2}{2\sigma^2}\right)$$

Taking the negative log-likelihood gives

$$-\log p(y_i \mid \mathbf{x}_i; \phi) = \frac{(y_i - f_\phi(\mathbf{x}_i))^2}{2\sigma^2} + \text{const.}$$

So, for fixed variance σ^2, maximising likelihood is equivalent to minimising the squared error, and over the dataset this leads to **MSE**.

Heteroscedastic Regression

In **heteroscedastic regression**, the observation noise is allowed to depend on the input. This contrasts with the **homoscedastic** case, where one fixed variance σ^2 is used for all samples.

We assume

$$y_i \mid \mathbf{x}_i \sim \mathcal{N}\big(\mu_\phi(\mathbf{x}_i),\ \sigma^2_\phi(\mathbf{x}_i)\big)$$

so the model predicts both

- a **mean** $\mu_\phi(\mathbf{x}_i)$, which is the expected target value
- a **variance** $\sigma^2_\phi(\mathbf{x}_i)$, which represents predictive uncertainty

The corresponding negative log-likelihood for one sample becomes

$$\mathcal{L}_i = \frac{\big(y_i - \mu_\phi(\mathbf{x}_i)\big)^2}{2\,\sigma^2_\phi(\mathbf{x}_i)} + \frac{1}{2}\log \sigma^2_\phi(\mathbf{x}_i) + \text{const.}$$

Interpreting Heteroscedastic Loss

The loss has two important terms:

- $\frac{(y_i - \mu_\phi(\mathbf{x}_i))^2}{2\,\sigma_\phi^2(\mathbf{x}_i)}$: large residuals are penalised, but less strongly when the model predicts high uncertainty
- $\frac{1}{2}\log \sigma_\phi^2(\mathbf{x}_i)$: prevents the model from making the variance arbitrarily large everywhere

Compared with MSE, this loss lets the model express **input-dependent uncertainty**: noisy or ambiguous inputs can receive higher variance, while reliable inputs can receive lower variance.

In practice, one often predicts $\log \sigma_\phi^2(\mathbf{x}_i)$ instead of $\sigma_\phi^2(\mathbf{x}_i)$ directly to ensure positivity and improve numerical stability.

Classification

In **classification**, the target is a discrete label, so the model should output **class probabilities**.

Two common cases are:

- **Binary classification**: $y \in \{0, 1\}$ and the model predicts one probability $\hat{y} = p(y = 1 \mid \mathbf{x})$
- **Multiclass classification**: $y \in \{1, \ldots, C\}$ and the model predicts a vector $\hat{\mathbf{y}}$ with $\sum_{c=1}^{C} \hat{y}_c = 1$

In practice, probabilities are often obtained from **logits** (raw scores before converting them to probabilities):

- **Sigmoid** for binary classification
- **Softmax** for multiclass classification

Once the model outputs probabilities, we need a loss that rewards high probability for the true class and penalises confident wrong predictions.

Sigmoid and Softmax

Neural networks often first produce **logits** z or $\mathbf{z}$, that is, raw scores before normalisation.

Sigmoid is used for binary classification:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

It maps one logit to a probability in $(0, 1)$, interpreted as $p(y = 1 \mid \mathbf{x})$.

Softmax is used for multiclass classification:

$$\hat{y}_c = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}}, \qquad c = 1, \ldots, C$$

It maps a vector of logits to class probabilities that are all non-negative (ensured by the exponential) and sum to 1 (denominator normalises the C outputs).

So, sigmoid gives one probability for two classes, while softmax distributes probability mass across C competing classes.

Cross-Entropy Loss

For classification with C classes, the model outputs a probability distribution $\hat{\mathbf{y}} = f_\phi(\mathbf{x}) \in [0,1]^C$ (e.g. via softmax), and cross-entropy is the negative log-likelihood of the true class.

The **cross-entropy** between the true label (one-hot $\mathbf{y}$) and the prediction is:

$$\ell(\hat{\mathbf{y}},\ \mathbf{y}) = -\sum_{c=1}^{C} y_c \log \hat{y}_c$$

Averaged over all N samples:

$$\mathcal{L}_{\mathrm{CE}}(\phi) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c} \log \hat{y}_{i,c}$$

Properties:

- Measures how well predicted probabilities match true labels
- For binary classification ($C = 2$):
  $\ell = -\big[y \log \hat{y} + (1-y)\log(1-\hat{y})\big]$
- Heavily penalises confident wrong predictions

Optimisation and Training

Training as Optimisation

Training a neural network means choosing parameters ϕ such that the model fits the training data well.

We express this through a loss function

$$\mathcal{L}(\phi)$$

and seek parameters that minimise it:

$$\phi^* = \arg\min_{\phi} \mathcal{L}(\phi)$$

Gradient Descent

We want to minimise a loss function $\mathcal{L}(\phi)$ with respect to the model parameters ϕ.

The basic idea of **gradient descent**:

- compute the gradient (vector of partial derivatives) of the loss with respect to the parameters

$$\nabla_{\phi}\mathcal{L}(\phi) = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \phi_1} \\ \vdots \\ \frac{\partial \mathcal{L}}{\partial \phi_P} \end{bmatrix}$$

- move the parameters a small step in the **negative** gradient direction with learning rate η:

$$\phi_{t+1} = \phi_t - \eta \nabla_{\phi}\mathcal{L}(\phi_t)$$

- repeat until the loss stops improving sufficiently

∇ = "nabla"
for one parameter: $\frac{\partial \mathcal{L}}{\partial \phi}$ instead of $\nabla_{\phi}\mathcal{L}(\phi)$

Local and Global Minima, Saddle Points

Gradient descent follows the local slope of the loss landscape, but this does **not** guarantee reaching the best possible solution.

Important cases are:

- **Global minimum**: the smallest value of $\mathcal{L}(\phi)$ over the whole parameter space
- **Local minimum**: a point that is better than all nearby points, but not necessarily best overall
- **Saddle point**: a point where the gradient can be small or zero, but some directions go up while others go down

For deep neural networks, the loss is usually **non-convex**, so there can be many local minima and saddle points.

Therefore, in practice we usually cannot be sure to find the **global minimum**; we aim for a solution with sufficiently low loss and good generalisation.

Forward and Backward Pass

Training a neural network is often organised into two passes through the model.

Forward pass:

- propagate the input $\mathbf{x}$ through all layers
- compute intermediate activations and the final prediction $\hat{\mathbf{y}}$
- evaluate the loss $\mathcal{L}(\phi)$

Backward pass:

- start from the loss
- propagate derivatives backward through the network using the chain rule
- compute the gradient of the loss with respect to all parameters, $\nabla_{\phi}\mathcal{L}(\phi)$

These gradients are then used by the optimiser to update the parameters.

Backpropagation

Backpropagation is the standard procedure that *implements the backward pass* and computes gradients in a neural network efficiently.

Main idea:

- apply the **chain rule** through the sequence of layers
- start with the derivative of the loss at the output layer
- propagate an error signal backward from layer to layer
- use these local derivatives to compute parameter gradients

If a network is a composition

$$f(\mathbf{x}) = f^{(L)} \circ \cdots \circ f^{(2)} \circ f^{(1)}(\mathbf{x}),$$

then backpropagation repeatedly applies the chain rule to obtain $\nabla_{\phi}\mathcal{L}(\phi)$.

The key advantage is that intermediate derivatives are **reused**, so all gradients can be computed much more efficiently than differentiating each parameter separately.

Algorithmic Differentiation

Algorithmic differentiation (AD) computes derivatives of a program by systematically applying the chain rule.

Key viewpoint:

- represent the computation as an **arbitrary computational graph**
- each node stores the information needed for differentiation
- traverse the graph in reverse to accumulate derivatives

In deep learning, these operations usually act on **tensors** rather than scalars.

Tensor operations can often be implemented efficiently and with substantial **parallelisation**, especially on GPUs.

Backpropagation is the special case of reverse-mode AD applied to neural networks.

Derivatives and the Backward Pass

The gradient $\frac{\partial \mathcal{L}}{\partial \phi}$ tells us how the loss changes with each parameter. Two observations motivate how backpropagation computes these efficiently:

Observation 1 (Forward pass): Each weight scales the activation of a source hidden unit. A small change to that weight is therefore amplified or attenuated by that activation. We run the network on each training example and *store all intermediate activations* — this is the **forward pass**. These stored values will be needed during the backward pass.

Observation 2 (Backward pass): A small change to any parameter ripples through all subsequent layers until it reaches the loss. To compute the effect, we need the sensitivity of every downstream layer — but these quantities are *shared* across all parameters in the same or earlier layers. We therefore compute them once, moving backwards through the network, reusing results at each step. This is the **backward pass**.

Stochastic Gradient Descent (SGD)

In high-dimensional non-convex problems, we usually cannot expect to find the **global optimum** of the loss function.

For ordinary gradient descent, the final solution is strongly influenced by the **starting point**.

Stochastic gradient descent (SGD) changes this picture by using a noisy estimate of the full gradient, typically computed from a mini-batch (random subset of the training data).

Intuitively, SGD adds some **noise** to each gradient step:

- the loss may not decrease at every single step
- the iterate may jiggle around instead of moving smoothly downhill
- the noise can help the optimisation jump from one valley to another

This can make SGD more effective than exact gradient descent in complicated loss landscapes.

Batches and Epochs

Mini-batch update rule: at each step, compute the gradient over a random subset B_t of indices:

$$\phi_{t+1} = \phi_t - \frac{\alpha}{|B_t|} \sum_{i \in B_t} \frac{\partial \ell_i}{\partial \phi}$$

where ℓ_i is the loss for sample i and α is the **learning rate**, governing step size (independent of local curvature).

Epochs & batch sizes:

- Batches are drawn *without* replacement; once all data are consumed, the dataset is reshuffled — one such full pass is one **epoch**.
- Batch size ranges from 1 (pure SGD) to the entire dataset (**full-batch GD** $\equiv$ standard gradient descent).

Alternative view: each iteration minimises a different mini-batch loss; the expected gradient nonetheless equals the full-data gradient.

Properties of SGD

- Each update improves fit to a subset of the data — steps are sensible even if not optimal.
- Drawing without replacement ensures all training examples contribute equally.
- Gradient over a mini-batch is cheaper to compute than over the full dataset.
- Noise in the updates can help *escape* local minima and saddle points (some batch is likely to have a significant gradient at any saddle).
- Empirically, SGD finds parameters that **generalise** well to unseen data.

Convergence & learning-rate schedules: Near the global minimum the model fits all data well, so gradients stay small regardless of batch — parameters stop changing appreciably. In practice a **learning-rate schedule** is used: start with a large α to explore broadly, then reduce it by a constant factor every N epochs to fine-tune once a good region is found.

Momentum

Momentum augments SGD by blending the current mini-batch gradient with the direction of the previous step:

$$\mathbf{m}_{t+1} \leftarrow \beta\, \mathbf{m}_t + (1-\beta) \sum_{i \in B_t} \frac{\partial \ell_i}{\partial \boldsymbol{\phi}}$$

$$\boldsymbol{\phi}_{t+1} \leftarrow \boldsymbol{\phi}_t - \alpha\, \mathbf{m}_{t+1}$$

where $\mathbf{m}_t$ is the momentum vector, $\beta \in [0, 1)$ controls the smoothing, and α is the learning rate.

Effect: the update is an exponentially weighted average of all past gradients.

- If gradients point in a consistent direction, contributions *accumulate* — effective step size grows.
- If the gradient direction keeps reversing (e.g. in a narrow valley), contributions *cancel* — oscillations are suppressed.

The net result is a smoother trajectory and faster progress along low-curvature directions.

Nesterov Accelerated Momentum

Key idea: standard momentum computes the gradient at the *current* parameters ϕ_t, then moves. Nesterov *first* projects where momentum alone would take us, then evaluates the gradient there:

1. **Lookahead step** — move by the current momentum: $\tilde{\phi}_t = \phi_t - \alpha\,\beta\,\mathbf{m}_t$
2. **Gradient at lookahead** — compute $\mathbf{g}_t = \sum_{i \in B_t} \frac{\partial \ell_i(\tilde{\phi}_t)}{\partial \phi}$

   $\equiv \sum_{i \in B_t} \left.\frac{\partial \ell_i}{\partial \phi}\right|_{\tilde{\phi}_t}$ ("evaluated at $\tilde{\phi}_t$")
3. **Momentum update:** $\mathbf{m}_{t+1} \leftarrow \beta\,\mathbf{m}_t + (1-\beta)\,\mathbf{g}_t$
4. **Parameter update:** $\phi_{t+1} \leftarrow \phi_t - \alpha\,\mathbf{m}_{t+1}$

Why it helps: by the time we apply the correction, we already know the slope *ahead* of us. If momentum is about to overshoot a minimum, the gradient at $\tilde{\phi}_t$ points back earlier, enabling a faster correction than waiting to discover the overshoot after landing.

Adaptive Gradient Methods

Problem: with a fixed learning rate, step size $\propto$ gradient magnitude — too large where steep, too small where flat. Hard to choose one α for anisotropic landscapes (i.e. where the loss is much steeper in some directions than others).

Idea: normalise each weight's gradient by its own magnitude, so every individual weight moves exactly α (all operations are *pointwise*, per parameter):

$$\mathbf{m}_{t+1} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{\phi}}, \quad \mathbf{v}_{t+1} = \mathbf{m}_{t+1}^{\odot 2}, \qquad \boldsymbol{\phi}_{t+1} \leftarrow \boldsymbol{\phi}_t - \alpha \cdot \frac{\mathbf{m}_{t+1}}{\sqrt{\mathbf{v}_{t+1}} + \epsilon}$$

($\odot 2$ = element-wise square; ϵ prevents division by zero.)
Only the *sign* per weight survives. However, this naive version oscillates without converging.

Adam

Adam (Adaptive Moment Estimation) fixes the oscillation by adding **momentum** to both statistics:

$$\mathbf{m}_{t+1} \leftarrow \beta\,\mathbf{m}_t + (1-\beta)\sum_{i\in B_t}\frac{\partial \ell_i}{\partial \boldsymbol{\phi}}$$

$$\mathbf{v}_{t+1} \leftarrow \gamma\,\mathbf{v}_t + (1-\gamma)\left(\sum_{i\in B_t}\frac{\partial \ell_i}{\partial \boldsymbol{\phi}}\right)^2$$

Bias correction (early iterates under-estimate, since history is zero-initialised):

$$\tilde{\mathbf{m}}_{t+1} = \frac{\mathbf{m}_{t+1}}{1-\beta^{t+1}}, \qquad \tilde{\mathbf{v}}_{t+1} = \frac{\mathbf{v}_{t+1}}{1-\gamma^{t+1}}$$

Update: $\boldsymbol{\phi}_{t+1} \leftarrow \boldsymbol{\phi}_t - \alpha \cdot \dfrac{\tilde{\mathbf{m}}_{t+1}}{\sqrt{\tilde{\mathbf{v}}_{t+1}} + \epsilon}$

Practical advantages: converges to the minimum; balances learning across all layers; less sensitive to the initial choice of α.

Initialisation

Before training, biases are set to zero and weights Ω_k are drawn from $\mathcal{N}(0, \sigma_\Omega^2)$. Here σ_Ω^2 is the *weight variance*; not to be confused with $\sigma(\cdot)$, the sigmoid activation function.

The choice of σ_Ω^2 is critical. Two failure modes:

- **Too small** ($\sigma_\Omega^2 \ll 1$): weighted sums produce outputs smaller than inputs; activations collapse layer by layer.
- **Too large** ($\sigma_\Omega^2 \gg 1$): weighted sums amplify inputs; activations explode layer by layer.

In both cases, activations can leave the range representable by finite-precision arithmetic.

Vanishing and Exploding Gradients

The same instability affects the **backward pass**. Each gradient step involves multiplying by Ω^T; poorly scaled weights cause gradient magnitudes to shrink or grow at every layer.

- **Vanishing gradients**: gradient magnitudes decay toward zero as they propagate back — early layers receive negligible updates and stop learning.
- **Exploding gradients**: gradient magnitudes grow without bound — parameter updates become unstable and training diverges.

The solution: choose σ_Ω^2 so that both activation and gradient variances remain approximately constant across all layers.

He Initialisation

For a layer with D_h inputs and ReLU activations, the variance of the next layer's pre-activations satisfies:

$$\sigma_{f'}^2 = \tfrac{1}{2} D_h \sigma_\Omega^2 \sigma_f^2$$

where σ_Ω^2 is the weight variance, σ_f^2 and $\sigma_{f'}^2$ are the pre-activation variances, and the $\frac{1}{2}$ accounts for ReLU clipping half the values to zero.

Setting $\sigma_{f'}^2 = \sigma_f^2$ (stable forward pass) gives:

$$\sigma_\Omega^2 = \frac{2}{D_h}$$

The backward pass gives analogously $\sigma_\Omega^2 = \frac{2}{D_{h'}}$. If $D_h \neq D_{h'}$, a compromise:

$$\sigma_\Omega^2 = \frac{4}{D_h + D_{h'}}$$

stabilises both directions simultaneously. This is known as **He initialisation**.

Performance

Training, Validation, and Test Set

To assess and tune a model, the dataset is split into three disjoint subsets:

- **Training set**: used to fit the parameters ϕ by minimising the loss.
- **Validation set**: used to evaluate the model *during development* — selecting architectures and hyperparameters. *Not* used for gradient updates.
- **Test set**: held out entirely; used *once* at the end to estimate real-world performance. Repeated use of the test set leads to overly optimistic estimates.

Typical splits: 70/15/15 or 80/10/10, but the right ratio depends on the total dataset size.

Generalisation and Sources of Error

The goal is not to minimise training loss, but **expected loss** on unseen data:

$$\mathcal{L}_{\text{test}}(\phi) = \mathbb{E}_{(\mathbf{x},y)\sim p_{\text{data}}}\left[\ell(y, f(\mathbf{x}, \phi))\right]$$

The gap $\mathcal{L}_{\text{test}} - \mathcal{L}_{\text{train}}$ is the **generalisation gap**. Sources of error:

- **Approximation error**: the model family cannot represent the true function (bias).
- **Estimation error**: finite training data means we learn a noisy estimate of the best-in-class parameters (variance).
- **Irreducible noise**: inherent stochasticity or label noise in the data that no model can eliminate.

Noise, Bias, and Variance

For a single output, the expected squared error of a model decomposes as:

$$\mathbb{E}\left[(y - \hat{y})^2\right] = \underbrace{\sigma_\epsilon^2}_{\text{noise}} + \underbrace{\left(\mathbb{E}[\hat{y}] - y^*\right)^2}_{\text{bias}^2} + \underbrace{\mathbb{E}\left[(\hat{y} - \mathbb{E}[\hat{y}])^2\right]}_{\text{variance}}$$

where y^* is the true value and σ_ϵ^2 is the irreducible noise variance.

- **Noise**: irreducible; inherent randomness in the data.
- **Bias**: systematic error — the model's average prediction is off.
- **Variance**: sensitivity to the particular training set drawn — model changes a lot across different training runs.

Reducing Variance

High variance means the model *overfits*: it fits the training data well but generalises poorly.

Strategies to reduce variance:

- **More training data**: variance $\propto 1/N$; more data averages out noise.
- **Regularisation**: add a penalty term to the loss, e.g. L2 (weight decay): $\tilde{\mathcal{L}}(\boldsymbol{\phi}) = \mathcal{L}(\boldsymbol{\phi}) + \lambda \|\boldsymbol{\phi}\|^2$
- **Early stopping**: halt training before the model starts memorising noise.
- **Dropout**: randomly zero out units during training, acting as an ensemble.
- **Ensembling**: average predictions of multiple independently trained models.
- **Reduce model capacity**: fewer parameters leave less room to memorise.

Reducing Bias

High bias means the model *underfits*: it cannot capture the structure in the data.

Strategies to reduce bias:

- **Increase model capacity**: more layers, wider layers, richer function class.
- **Train longer**: ensure the optimiser has converged.
- **Better architecture**: choose an inductive bias suited to the data (e.g. convolutions for images, attention for sequences).
- **Feature engineering**: provide more informative inputs.
- **Reduce regularisation**: a too-strong penalty can introduce bias.

Reducing bias often increases variance and vice versa — see the bias–variance trade-off.

Bias–Variance Trade-off

Total expected error $=$ noise $+$ bias^2 $+$ variance.

Changing model capacity affects the two controllable terms in opposite directions:

- **Low capacity** (underfit): high bias, low variance — the model is consistently wrong.
- **High capacity** (overfit): low bias, high variance — the model is right on average but unreliable.
- **Sweet spot**: intermediate capacity minimises total test error.

The optimal operating point is found by monitoring **validation loss** as a function of capacity or regularisation strength:

$$\phi^* = \arg\min_{\phi} \; \mathcal{L}_{\text{val}}(\phi)$$

Double Descent

Classical theory predicts test error forms a U-curve: decrease then increase with capacity. Modern overparameterised models (parameters $\gg$ data) exhibit a second descent:

1. **Classical regime**: test error decreases with capacity up to the interpolation threshold.
2. **Interpolation threshold**: the model is just large enough to fit the training data exactly — test error peaks (the "double descent peak").
3. **Modern regime**: beyond the threshold, further capacity *reduces* test error again, as the model finds smoother interpolants.

Deep neural networks typically operate in the modern regime. This explains why very large models can generalise well despite overfitting in the classical sense.

[1, 11]

Capacity

Capacity (or complexity) refers to the richness of functions a model can represent.

Measures of capacity:

- **Number of parameters**: the most common proxy.
- **Vapnik–Chervonenkis dimension**: the largest number of points the model can *shatter* — i.e. correctly classify under *every possible* labelling. A model with VC dimension h can represent 2^h distinct binary labellings of h points. Higher VC dimension $\Rightarrow$ richer function class $\Rightarrow$ more capacity.
- **Rademacher complexity**: expected ability to fit random labels.

Capacity is also affected by:

- Architecture choices (depth, width, skip connections)
- Regularisation (effectively reduces capacity)
- Activation functions and normalisation layers

Hyperparameter Search

Hyperparameters (learning rate, batch size, architecture depth, regularisation λ, ...) are *not* learned by gradient descent — they must be chosen beforehand.

Common search strategies:

- **Grid search**: evaluate all combinations on a discrete grid. Exhaustive but expensive.
- **Random search**: sample configurations randomly. Often more efficient than grid search in high dimensions (most hyperparameters matter little).
- **Bayesian optimisation**: maintain a probabilistic surrogate model over the objective; select the next configuration by maximising an acquisition function.

All strategies evaluate performance on the **validation set**; the test set must remain unseen.

Bayesian Optimisation

Goal: minimise $\mathcal{L}_{\text{val}}(\boldsymbol{\lambda})$ over hyperparameters $\boldsymbol{\lambda}$ with as few evaluations as possible (each evaluation = a full training run).

Algorithm (repeat until budget exhausted):

1. Fit a **surrogate model** — typically a Gaussian process — to all previous $(\boldsymbol{\lambda}_i, \mathcal{L}_i)$ observations. The GP provides a mean prediction *and* uncertainty estimate at every $\boldsymbol{\lambda}$.
2. Maximise an **acquisition function** to propose the next $\boldsymbol{\lambda}$:
   - *Expected Improvement* (EI): expected gain over the current best.
   - *Upper Confidence Bound* (UCB): mean $+\kappa\cdot$std; κ controls explore/exploit.
3. Evaluate the model at the proposed $\boldsymbol{\lambda}$; record $\mathcal{L}_{\text{val}}$.

The acquisition function balances **exploration** (high uncertainty regions) with **exploitation** (regions near the current best). Requires far fewer evaluations than grid or random search.

Cross-Validation

When data is scarce, a fixed validation split wastes examples and gives a noisy estimate of generalisation performance.

Cross-validation repeatedly splits the data, training on one part and validating on the other, then averages performance:

$$\hat{\mathcal{L}} = \frac{1}{K} \sum_{k=1}^{K} \mathcal{L}_{\text{val}}^{(k)}$$

Benefits:

- Every example is used for both training and validation (across folds).
- The estimate of generalisation error has lower variance.

The most common variant is **k-fold cross-validation**.

k-Fold Cross-Validation

1. Partition the dataset into K equally-sized folds $F_1, \ldots, F_K$.
2. For each fold $k = 1, \ldots, K$:
   - Train on all folds *except* F_k.
   - Evaluate on F_k to get $\mathcal{L}_{\text{val}}^{(k)}$.
3. Report $\hat{\mathcal{L}} = \frac{1}{K} \sum_{k=1}^{K} \mathcal{L}_{\text{val}}^{(k)}$.

- Typical choices: $K = 5$ or $K = 10$.
- **Leave-one-out** ($K = N$): least biased but most expensive.
- After selecting hyperparameters, retrain on *all* data before final evaluation.

Curse of Dimensionality

To *densely sample* a D-dimensional input space with resolution r requires

$$N \propto r^D$$

data points — exponential in the number of dimensions.

Consequences:

- In high dimensions, data becomes extremely sparse; nearest neighbours are no longer meaningfully "near".
- The volume of space grows so fast that any finite dataset covers only a negligible fraction.
- Models that rely on local smoothness (e.g. k-NN, kernel methods) degrade rapidly with dimension.

Deep networks partially escape this by learning **low-dimensional representations** — the data may lie on a low-dimensional manifold even though the raw input is high-dimensional.

Regularisation

Combating Overfitting

A model that fits the training data perfectly may still generalise poorly — it has *overfit* to noise or specifics of the training set. **Regularisation** refers to any technique that reduces this generalisation gap.

Two broad categories:

- **Explicit regularisation**: directly modifies the objective by adding a penalty term that discourages complex or large-magnitude parameters (e.g. L2 / weight decay).
- **Implicit regularisation**: an unintended but beneficial side-effect of the optimisation algorithm itself (e.g. the discretisation of gradient descent, mini-batch noise in SGD).

Beyond these, a range of **heuristic methods** also improve generalisation: early stopping, ensembling, dropout, noise injection, transfer learning, and data augmentation.

Explicit Regularisation

Explicit regularisation adds a penalty term directly to the loss to discourage undesirable parameter values:

$$\phi^* = \arg\min_{\phi} \left[\sum_{i=1}^{I} \ell_i[\mathbf{x}_i, y_i] + \lambda \cdot g[\phi] \right]$$

where $g[\phi] \geq 0$ is a penalty on the parameters and $\lambda \geq 0$ controls the strength.

Probabilistic interpretation: the penalty corresponds to placing a prior $\Pr(\phi)$ on the parameters. Minimising the regularised loss is equivalent to *maximum a posteriori* (MAP) estimation:

$$\phi^*_{\text{MAP}} = \arg\max_{\phi} \sum_{i} \log \Pr(y_i \mid \mathbf{x}_i, \phi) + \log \Pr(\phi)$$

The prior $\Pr(\phi)$ encodes our belief about good parameter values before seeing any data.

L2 Regularisation

The most common choice is **L2 regularisation**, which penalises the sum of squared parameters:

$$\boldsymbol{\phi}^* = \arg\min_{\boldsymbol{\phi}} \left[\sum_{i=1}^{I} \ell_i[\mathbf{x}_i, y_i] + \lambda \sum_{j} \phi_j^2 \right]$$

Also known as:

- **Tikhonov regularisation** (general form)
- **Ridge regression** (applied to linear models)
- **Frobenius norm regularisation** (when applied to matrices)
- **Weight decay** (when applied to neural network weights)

Corresponds to a **Gaussian prior** $\Pr(\boldsymbol{\phi}) \propto \exp(-\lambda \|\boldsymbol{\phi}\|^2)$ on the parameters.

Weight Decay in Neural Networks

For neural networks, L2 regularisation is applied to the **weights** (not the biases) and is called **weight decay**.

Effect: smaller weights mean the output varies less with respect to each activation — the learned function is smoother. In the extreme limit $\lambda \to \infty$, all weights are forced to zero and the network produces a constant output determined entirely by the biases.

Why this can improve test performance:

- **Overfitting**: the network trades a perfect fit for smoothness — variance decreases at the cost of slightly increased bias.
- **Overparameterisation**: in regions with no training data, weight decay favours smooth interpolation between nearby training points rather than arbitrary extrapolation.

Increasing λ reduces fit accuracy but improves smoothness; the optimal λ is chosen on a validation set.

Implicit Regularisation

An intriguing recent finding: neither gradient descent nor SGD moves *neutrally* toward a minimum — each exhibits a *preference* for certain solutions over others. This is known as **implicit regularisation**.

The continuous (infinitesimal step size) version of gradient descent follows:

$$\frac{d\phi}{dt} = -\frac{\partial \mathcal{L}}{\partial \phi}$$

Discretisation with step size α introduces a deviation from this continuous path, effectively optimising a *modified* loss.

Implicit Regularisation: Gradient Descent

Discrete gradient descent with step size α is equivalent to continuously minimising a **modified loss**:

$$\tilde{\mathcal{L}}_{\mathrm{GD}}[\boldsymbol{\phi}] = \mathcal{L}[\boldsymbol{\phi}] + \frac{\alpha}{4}\left\|\frac{\partial \mathcal{L}}{\partial \boldsymbol{\phi}}\right\|^2$$

The extra term **repels** the trajectory from steep regions (large gradient norm). It does not shift the minima themselves (where the gradient is zero), but it does change the path taken and can cause convergence to a *different* minimum.

Practical implication: full-batch gradient descent tends to generalise better with *larger* step sizes, consistent with stronger implicit smoothing.

Implicit Regularisation: SGD

A similar analysis for SGD yields a modified loss with an additional term:

$$\tilde{\mathcal{L}}_{\mathrm{SGD}}[\phi] = \mathcal{L}[\phi] + \frac{\alpha}{4}\left\|\frac{\partial \mathcal{L}}{\partial \phi}\right\|^2 + \frac{\alpha}{4B}\sum_{b=1}^{B}\left\|\frac{\partial \mathcal{L}_b}{\partial \phi} - \frac{\partial \mathcal{L}}{\partial \phi}\right\|^2$$

where $\mathcal{L}_b$ is the loss for mini-batch b.

The extra term is the **variance of the per-batch gradients**: SGD implicitly favours solutions where all batches *agree* on the gradient direction.

Practical implications:

- SGD generalises better than full-batch gradient descent.
- Smaller batches introduce more variance, strengthening the implicit regularisation.

Early Stopping

Early stopping halts training before the loss has fully converged.

Why it works:

- Weights are initialised near zero; they simply do not have time to grow large — similar effect to L2 regularisation.
- Reduces effective model complexity: the model captures the coarse structure of the data but has not yet had time to overfit to noise.

Hyperparameter: number of training steps T after which learning is halted. Unlike most hyperparameters, this requires only a *single training run*: monitor validation loss every T steps, save parameters at each checkpoint, and select the checkpoint with the best validation performance.

Ensembling

Ensembling trains multiple models independently and combines their predictions, reliably improving test performance at the cost of additional compute.

Combining predictions:

- Regression: mean (or median for robustness) of outputs.
- Classification: mean of pre-softmax logits, or majority vote.

Strategies to build diverse models:

- Different random weight initialisations.
- **Bagging** (bootstrap aggregating): re-sample the training set with replacement for each model; outliers appear less frequently, smoothing the fitted function.
- Different hyperparameters or entirely different model families.

The assumption is that model errors are *independent* and cancel when averaged.

Dropout

Dropout randomly clamps a subset of hidden units (typically 50%) to zero at each SGD iteration.

Effects:

- Prevents the network from co-adapting to any specific hidden unit — representations must be redundant and robust.
- Encourages smaller weight magnitudes, since the influence of a unit that may be absent is bounded.
- Eliminates spurious "kinks" in the function far from the training data that happen to arise during training.

Interpretation: can be viewed as training an exponentially large ensemble of sub-networks that share weights, and averaging their predictions at test time.

At **test time**, all units are active and weights are scaled by the keep probability.

Applying Noise

Adding noise during training encourages robustness. Common variants:

- **Input noise**: add random perturbations to $\mathbf{x}$ during training. Equivalent to a regulariser that penalises the derivative of the output with respect to the input — the function is encouraged to be locally flat. The extreme case is **adversarial training**, which injects worst-case perturbations.
- **Weight noise**: add noise to the parameters ϕ. Encourages the network to converge to minima in wide, flat regions where individual weight changes matter little.
- **Label smoothing**: instead of one-hot targets (certainty of 1 for the true class), use a soft target: probability $1 - \rho$ on the true class and $\rho/(C-1)$ on each other class. Prevents overconfident predictions and improves generalisation.

Bayesian Inference

Maximum likelihood selects a single parameter vector ϕ^*, ignoring uncertainty. The **Bayesian approach** instead computes a full posterior distribution:

$$\Pr(\phi \mid \{\mathbf{x}_i, y_i\}) = \frac{\prod_i \Pr(y_i \mid \mathbf{x}_i, \phi)\ \Pr(\phi)}{\int \prod_i \Pr(y_i \mid \mathbf{x}_i, \phi)\ \Pr(\phi)\, d\phi}$$

Predictions are made by *integrating out* the parameters — effectively an infinite weighted ensemble:

$$\Pr(y \mid \mathbf{x}, \{\mathbf{x}_i, y_i\}) = \int \Pr(y \mid \mathbf{x}, \phi)\ \Pr(\phi \mid \{\mathbf{x}_i, y_i\})\, d\phi$$

Limitation: the posterior is intractable for large neural networks. All practical methods require approximations (e.g. variational inference, Monte Carlo dropout), adding considerable complexity.

Transfer Learning and Multi-task Learning

Transfer learning: pre-train on a large secondary task (where data are plentiful), then adapt to the target task.

- Replace the final layer(s) with task-specific heads.
- Either *fine-tune* the whole network or freeze the backbone and train only the new layers.
- Principle: the backbone builds a rich internal representation that transfers across tasks, and initialises parameters in a good region of parameter space.

Multi-task learning: train a single network to solve several tasks simultaneously (e.g. scene segmentation, depth estimation, and captioning from the same image). Shared representations benefit all tasks and act as a form of regularisation.

Self-supervised learning: when no labelled secondary task exists, generate "free" labels from the data itself — e.g. predict masked patches (generative) or identify transformed pairs (contrastive) — then fine-tune for the target task.

Augmentation

Data augmentation artificially expands the training set by applying label-preserving transformations to existing examples.

Common transforms by modality:

- **Images**: rotation, horizontal flip, crop, colour jitter, blur.
- **Text**: synonym substitution, back-translation.
- **Audio**: frequency-band amplification/attenuation, time-stretching.

Effect: the model sees more diverse inputs and cannot simply memorise fixed examples. It acts as a regulariser by encouraging the learned function to be invariant to the chosen transformations.

Augmentation is inexpensive, broadly applicable, and consistently improves generalisation — it is standard practice in almost all modern training pipelines.

References I


Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal.
Reconciling modern machine-learning practice and the classical bias–variance trade-off.
Proceedings of the National Academy of Sciences, 116(32):15849–15854, 2019.


Richard E. Bellman.
Dynamic Programming.
Princeton University Press, 1957.


Leo Breiman.
Bagging predictors.
Machine Learning, 24(2):123–140, 1996.


Stuart Geman, Elie Bienenstock, and René Doursat.
Neural networks and the bias/variance dilemma.
Neural Computation, 4(1):1–58, 1992.

References II


Ian Goodfellow, Yoshua Bengio, and Aaron Courville.
Deep Learning.
The MIT Press, 2016.


Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy.
Explaining and harnessing adversarial examples.
In *International Conference on Learning Representations (ICLR)*, 2015.


Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun.
Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification.
In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015.


Diederik P. Kingma and Jimmy Ba.
Adam: A method for stochastic optimization.
arXiv preprint arXiv:1412.6980, 2014.

References III


Ron Kohavi.
A study of cross-validation and bootstrap for accuracy estimation and model selection.
In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1137–1143, 1995.


Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner.
Gradient-based learning applied to document recognition.
Proceedings of the IEEE, 86(11):2278–2324, 1998.


Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever.
Deep double descent: Where bigger models and more data hurt.
In *International Conference on Learning Representations (ICLR)*, 2020.

References IV


Yurii Nesterov.
A method for solving the convex programming problem with convergence rate $O(1/k^2)$.
Doklady Akademii Nauk SSSR, 269:543–547, 1983.


Simon J.D. Prince.
Understanding Deep Learning.
The MIT Press, 2026.


David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams.
Learning representations by back-propagating errors.
Nature, 323:533–536, 1986.

References V


Samuel L. Smith and Quoc V. Le.
A Bayesian perspective on generalization and stochastic gradient descent.
In *International Conference on Learning Representations (ICLR)*, 2018.


Jasper Snoek, Hugo Larochelle, and Ryan P. Adams.
Practical Bayesian optimization of machine learning algorithms.
In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25, 2012.


Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov.
Dropout: A simple way to prevent neural networks from overfitting.
Journal of Machine Learning Research, 15:1929–1958, 2014.

References VI


Vladimir N. Vapnik.
The Nature of Statistical Learning Theory.
Springer, 1995.